
LUCRAREA DE LABORATOR №3

STRUCTURI DINAMICE DE DATE. LISTE LINIARE

Scopul lucrării:

1. Studiarea operațiilor cu structuri de date alocate dinamic: crearea, adăugarea, ștergerea și parcurgerea.
2. Realizarea operațiilor cu structuri dinamice de date: stiva, coada, lista circulară, lista dublu înălțuită.
3. Studiarea mecanismului de prelucrare a elementelor structurilor dinamice de date.
4. Proiectarea algoritmilor cu structuri dinamice de date.

Indicații metodice:

1. Datele pot fi introduse de la tastatură pe unul sau mai multe rânduri.
2. Datelor de tip logic li se pot atribui valori în secțiunea definirii constantelor sau prin instrucțiunea de atribuire.
3. În regim dialog (interactiv) de afișat mesaje sugestive despre natura datelor citite de la tastatură.
4. De afișat datele de ieșire prin antet.

Structurile de date alocate dinamic sunt structuri ale căror componente se alocă în cursul execuției programului (dinamic). Structurile dinamice sunt mult mai flexibile decât cele statice și extrem de avantajoase acolo unde este necesară această flexibilitate. Există mai multe tipuri de structuri dinamice: *liniare*, *arborescente* și *rețea*.

Structurile liniare se mai numesc și **liste**.

Lista liniară este o structură de date logică, cu date omogene, în care fiecare element are un succesor și un predecesor, exceptând primul element, care nu are decât succesor, și ultimul element care nu are decât predecesor.

O **listă liniară** este formată din noduri. Fiecare nod reprezintă un articol cu 2 câmpuri:

- un câmp informațional, care poate fi de orice tip;
- un câmp de legătură, care va fi de tip reper.

Câmpurile de legătură a tuturor nodurilor, în afară de ultimul nod, conțin adresele nodurilor următoare. Adresa primului nod se află într-o variabilă reper (pointer), alocată static, iar câmpul de legătură a ultimului nod conține valoarea `Nil`, astfel specificându-se sfârșitul listei.

Variantă Pascal	Variantă C/C++
Type Nod = Record inf: Integer; urm: ^Nod; End; Var Lista: ^Nod;	<pre>struct nod { int inf; nod *urm; };</pre>

Asupra listelor liniare se fac următoarele operații:

- crearea listei vide;
- inserarea unui nod (la începutul listei; la sfârșitul listei; după elementul P; înaintea elementul P);
- ștergerea unui nod;
- traversarea nodurilor listei liniare.

În cazul în care se impun anumite restricții asupra operațiilor de inserare, ștergere sau consultare a elementelor unei liste liniare, se obțin câteva cazuri particulare de liste mult folosite în practică: stiva, coada, lista circulară și lista dublu înlănțuită.

Stiva este o listă pentru care singurele operații permise sunt:

- adăugarea unui element în vârful stivei;
- eliminarea sau modificarea ultimului element introdus în stivă.

Stiva funcționează pe principiul LIFO (Last In First Out) - "ultimul intrat primul ieșit".

O **coadă** este o listă pentru care toate inserările sunt făcute la unul din capete, iar toate ștergerile (modificările) la celălalt capăt. Coada funcționează pe principiul FIFO (First In First Out) - "primul intrat primul ieșit".

O **listă circulară** (simplu înlănțuită) este o listă liniară simplu înlănțuită în care câmpul de adresă al ultimului nod reține adresa primului nod.

Listele liniare dublu înlănțuite sunt liste liniare în care fiecare element (nod) are două câmpuri de adresă: al elementului anterior și al elementului următor. Fiecare nod are două referințe, spre succesorul respectiv predecesorul lui.

PROBLEME REZOLVATE:

1. Deservirea clienților.

Se consideră un ghișeu la care se așteaptă la coadă pentru a fi serviți mai mulți clienți. Să se scrie un program care prelucrează comenzi de forma: I (intrarea la coadă), E (ieșirea de la coadă în urma servirii de la ghișeu), L (listarea persoanelor ce se află la coadă) și S (sfârșitul programului). Clienții sunt codificați numeric.

Limbajul de programare C

```

1.  #include <stdio.h>
2.  #include <conio.h>
3.  #include <stdlib.h>
4.  #include <iostream>
5.  using namespace std;
6.  struct nod {int cod;
7.      nod *urm;};
8.  nod *prim, *ultim, *p;
9.  void intrare()
10. {
11.     int codul;
12.     cout<<"Intra clientul: ";
13.     cin>>codul;
14.     if (prim==NULL)
15.         //coada vida, se adauga primul element
16.         {
17.             prim=new nod; ultim=prim;
18.             prim->cod=codul; prim->urm=NULL;
19.         }
20.     else
21.     {
22.         nod *p=new nod; p->cod=codul;
23.         p->urm=NULL; ultim->urm=p;  ultim=p; }
24. }
25. void iesire () //sterge primul
26. {
27.     if (prim==NULL)
28.     { cout<<"Nu mai sunt clienti";
29.       getch();
30.     }
31.     else
32.     { cout<<"Iese clientul "<<prim->cod
33.       <<endl;
34.       nod *p=prim; prim=prim->urm; delete p;
35.     } getch();
36. }
37. void afisare ()
38. {
39.     if (prim==NULL)
40.     { cout<<"Nu e nimeni la coada";}
41.     else
42.     { cout<<"Clientii de la coada: ";
43.       for (nod *p=prim; p!=NULL; p=p->urm)
44.         cout<<p->cod<<" "; cout<<endl;
45.     } getch();
46. }
47. int main ()
48. {
49.     char comanda;
50.     prim=NULL;
51.     do
52.     {
53.         cout<<"Comanda: "; cin>>comanda;
54.         switch (comanda)
55.         {
56.             case 'i': intrare(); break;
57.             case 'e': iesire(); break;
58.             case 'l': afisare(); break;
59.             default: {cout<<"Comanda incorecta"<<
60.                      endl;}
61.         }
62.     } while (comanda!='s');
63. }
64.

```

Limbajul de programare Pascal

```

Program Lista;
Type
    Nod = Record
        cod: integer;
        urm: ^Nod;
    end;
Var comanda: char;
    prim, ultim, p:^Nod;
procedure intrare;
Var codul:integer;
begin
    write('Intra clientul: '); Readln(codul);
    if prim=nil then
    begin
        New(prim);
        prim^.cod:=codul; prim^.urm:=nil;
        ultim:=prim;
    end
    else
    begin
        New(p); p^.cod:=codul; p^.urm:=nil;
        ultim^.urm:=p; ultim:=p;
    end;
end;
procedure iesire;
begin
    if prim=nil then
        writeln('Nu mai sunt clienti')
    else
    begin
        writeln('Iese clientul ', prim^.cod);
        p:=prim; prim:=prim^.urm;

        dispose(p);
    end;
end;
procedure afisare;
begin
    if prim=nil then
        writeln('Nu e nimeni la coada')
    else
    begin
        write('Clientii de la coada: ');
        p:=prim;
        while p<>nil do
        begin
            write(p^.cod,' ');
            p:=p^.urm;
        end;
    end;
end;
Begin
    prim:= nil;
    repeat
        write('Comanda:');
        readln(comanda);
        case comanda of
            'i': intrare;
            'e': iesire;

            'l': afisare;
        else
            writeln('Comanda incorecta');
        end;
    until comanda='s'
End.

```

2. Evaluarea expresiei în forma postfixată.

Să se scrie o funcție pentru evaluarea unei expresii postfixate sub forma unui șir de caractere.

Exemple de expresii postfixate:

53- echivalentă cu $5-3 = 2$
 31+2* echivalentă cu $(3+1)*2 = 8$
 73-31-* echivalentă cu $(7-3)*(3-1) = 8$
 4853-+23*- echivalentă cu $4+8*(5-3)-2*3 = 14$ $4+8*2-6=4+16-6$

Algoritmul de evaluare a unei expresii în formă postfixată:

1. Cât timp există elemente de citit

1.1 citește un element

1.2 dacă elementul este o valoare

1.2.1 pune elementul în stivă altfel (elementul este un operator)

1.2.2 extrage 2 operanzi din stivă

1.2.3 dacă nu există 2 operanzi în stivă

altfel

eroare: forma postfixată nu este corectă

1.2.4 evaluează rezultatul aplicării operatorului asupra celor doi operanzi

1.2.5 pune rezultatul în stivă

2. Dacă există o singură valoare în stivă

2.1 afișează valoarea ca rezultat final al evaluării expresiei

altfel

eroare: forma postfixată nu este corectă

Limbajul de programare C

```
1. #include <stdio.h>
2. #include <conio.h>
3. #include <stdlib.h>
4. #include <iostream>
5. #include <string.h>
6. #include <cstring>
7. using namespace std;
8. struct nod {int info;
9.             nod *urm;};
10. void intrare(int n, nod *&St)
11. //se adauga element in stiva
12. {
13.     nod *p=new nod; p->info=n;
14.     p->urm=St; St=p;
15. }
16. void iesire (nod *&St)
17. //sterge primul din stiva
18. {
19.     nod *p=St; St=St->urm; delete p;
20. }
21. int main ()
```

Limbajul de programare Pascal

```
Program Expresie;
Type
    Stack = ^Nod;
    Nod = Record
        info: integer;
        urm: Stack;
    end;
Var Ex: string;
    St: Stack;
    op1, op2, rez, i: integer;
procedure AddSt(c: integer; Var S: Stack);
Var P: Stack;
begin
    New(P);
    P^.info:=c;
    P^.urm:=S;
    S:=P;
end;
procedure DeleteSt(Var S: Stack);
Var P: Stack;
begin
```

22. {	P:=S; S:=S^.urm;
23. char Ex[256];	dispose(p);
24. int op1,op2, rez=0, i;	end;
25. nod *St;	Begin
26. St=NULL;	St:= Nil;
27. cout<<"Introdu expresia postfixata: ";	write('Introdu expresia postfixata:');
28. cin>>Ex;	readln(Ex);
29. for (i=0;i<strlen(Ex);i++)	rez:=0;
30. { if (Ex[i]>='0' && Ex[i]<='9')	for i:=1 to length(Ex) do
31. { AddSt(int(Ex[i])-int('0'),St); }	begin
32. else	if Ex[i] in ['0'..'9'] then
33. {	AddSt(Ord(Ex[i])-Ord('0'),St)
34. if (St==NULL)	else
35. {cout<<"forma postfixata incorecta";	begin
36. break; }	if St=Nil then
37. else	begin
38. { op1=St->info; DeleteSt(St); }	write('forma postfixata incorecta');
39. if (St==NULL)	Exit;
40. {cout<<"forma postfixata incorecta";	end
41. break; }	else
42. else	begin
43. { op2=St->info; DeleteSt(St); }	op1:=St^.info; DeleteSt(St);
44. switch (Ex[i])	end;
45. {	if St=Nil then
46. case '+': rez=op2+op1; break;	begin
47. case '-': rez=op2-op1; break;	write('forma postfixata incorecta');
48. case '*': rez=op2*op1; break;	Exit;
49. default: {cout<<"forma incorecta";}	end
50. }	else
51. AddSt(rez,St);	begin
52. }	op2:=St^.info; DeleteSt(St);
53. }	end;
54. if (St->urm!=NULL)	Case Ex[i] of
55. cout<<"forma postfixata incorecta";	'+' : rez:=op2+op1;
56. else	'-' : rez:=op2-op1;
57. cout<<"Rez="<<St->info;	'*' : rez:=op2*op1;
58. }	else
59.	begin
60.	write('forma postfixata incorecta');
61.	Exit;
62.	end;
63.	end;
64.	AddSt(rez,St);
65.	end;
66.	end;
67.	if St^.urm<>Nil then
68.	write('forma postfixata incorecta')
69.	else write('Rez=',St^.info);
70.	End.

Probleme propuse:

Elaborați algoritmi pentru rezolvarea următoarelor probleme (rezolvați doar o singură problemă):

1. Un număr de N copii stau așezați în cerc și joacă următorul joc: se numără începând de la copilul X un număr de Z copii, cel care este cel cu numărul Z este eliminat, după care se continuă numărătoarea de la următorul. Jocul se termină atunci când va rămâne doar un singur copil în cerc. Să se scrie un program care să simuleze aceste operații ale jocului tipărint numele copiilor care sunt eliminați în ordinea operației și la final pe cel câștigător.

2. Se dă o listă liniară simplu înlănțuită în care fiecare nod reține o literă. Să se elaboreze un program care depistează dacă cuvântul format prin alăturarea literelor citite este sau nu palindrom (se obține același rezultat dacă cuvântul se citește direct sau invers).
Exemplu: lista următoare conține un cuvânt palindrom: $a \rightarrow b \rightarrow c \rightarrow b \rightarrow a$
3. Se citește dintr-un fișier două numere foarte mari. Să se elaboreze un program pentru prelucrarea numerelor, folosind următoarele subprograme:
 - a. crearea listelor care conțin cifrele a două numere foarte mari;
 - b. calcularea sumei, a diferenței și produsului numerelor;
 - c. compararea a două numere (subprogramul trebuie să verifice dacă cele două numere sunt egale, iar dacă nu sunt egale, să precizeze care număr este mai mare).
4. Se citește dintr-un fișier două numere foarte mari. Să se elaboreze un program pentru prelucrarea numerelor, folosind următoarele subprograme:
 - a. crearea listelor care conțin cifrele a două numere foarte mari;
 - b. verificarea numărului dacă are un număr par sau un număr impar de cifre, fără să se numere cifrele numărului;
 - c. determinarea numărului de cifre ale unui număr;
 - d. verificarea numărului dacă este palindrom.
5. Sunt date șiruri de caractere finite alcătuite din paranteze: (), { }, []. Șirul este considerat corect, dacă el este format conform următoarelor reguli:
 - a. șirul vid este corect;
 - b. dacă A este un șir corect, atunci (A), [A], {A} – șiruri corecte;
 - c. dacă A și B – șiruri corecte, atunci AB – șir corect.
 Să se elaboreze un program care verifică dacă șirul de caractere citit de la tastatură este corect. *Exemplu:* șirurile (), [], {}, [()], ({()}), ([{}])({{})) – corecte, iar șirurile [], ([, {{ – incorecte.
6. Se citește un text, caracter cu caracter, din șirul S. Șirul de caractere este citit de la tastatură și poate conține orice caractere. Să se elaboreze un program care creează o listă liniară dublu înlănțuită cu caracterele distincte din text în ordine alfabetică împreună cu frecvențele lor de apariție în cadrul textului și afișează conținutul listei pe ecran (pe fiecare rând se va scrie câte un caracter distinct și frecvența sa de apariție, separate printr-un spațiu).
7. Se citește un șir de numere de la tastatură. Să se elaboreze un program care ordonează crescător un șir de numere, cu ajutorul a două stive. (*Indicație.* Se folosesc două stive. Se scrie un număr în Stiva 1. Se citește un număr. Dacă numărul din vârful Stivei 1 este mai mare decât numărul citit, atunci noul număr se adaugă la Stiva 1; altfel, se descarcă din Stiva 1 în Stiva 2 numere până când în vârful Stivei 1 ajunge un număr mai mare decât numărul citit, se adaugă la Stiva 1 numărul citit, și se încarcă în Stiva 1 numerele din Stiva 2).

8. Pe o masă se găsesc n stive de farfurii colorate diferite. Se cere să se realizeze o singură stivă din cele n în așa fel încât să alterneze culorile farfuriilor (în număr egal). *Exemplu:* trei stive de culorile roșu, galben și albastru cu câte 6, 3 și respectiv 5 farfurii, va rezultă o stivă de 9 farfurii astfel: roșu, galben, albastru, roșu, galben, albastru, roșu, galben și albastru. Să se elaboreze un program care tipărește culorile stivei rezultat.
9. În două fișiere text F1.IN și F2.IN se găsesc 2 mulțimi A și respectiv B cu elemente numere naturale, separate prin spațiu.
 - a) Să se creeze câte o listă liniară simplă înlănțuită pentru memorarea elementelor din fiecare dintre cele 2 mulțimi.
 - b) Să se creeze o listă simplă înlănțuită care se memoreze reuniunea celor 2 mulțimi.
 - c) Să se creeze o listă simplă înlănțuită care se memoreze intersecția celor 2 mulțimi.
10. Să se creeze o listă dublu înlănțuită cu stocurile articolelor de mărfuri dintr-o firmă ordonată după codurile mărfurilor și să se actualizeze această listă (adăugarea la locul potrivit a unui articol, ștergerea și modificarea datelor unui articol ca urmare a intrărilor și ieșirilor de articole) urmată de listarea la cerere a tuturor articolelor.
11. Există două automate care eliberează bonuri de ordine pentru o coadă de așteptare. Pe fiecare bon de ordine este trecut numărul bonului și momentul la care a fost eliberat (exprimat în oră, minut și secundă). Bonurile au numere unice. Deservirea persoanelor se face în ordinea momentului de eliberare a bonurilor. Să se elaboreze un program care organizează o coadă de așteptare pe baza bonurilor emise de cele două automate.
12. Să se scrie un program care să administreze un parc de automobile (coadă). Informațiile relative la un automobil sunt: numărul de înmatriculare, marca, culoarea, numărul de locuri, puterea (în cai putere), tipul de carburant (benzină sau motorină). Programul să permită intrarea unei mașini, ieșirea unei mașini și listarea mașinilor.
13. Fișierul textual conține pe un singur rând o frază, alcătuită din cuvinte, între care apar ca separatori spațiul și virgula, fraza încheindu-se cu caracterul "punct". Să se elaboreze un program care creează o listă liniară simplă înlănțuită cu cuvintele frazei (câmpul de informație al fiecărui nod va fi de tip șir de caractere și va memora un cuvânt al frazei) și afișează cuvântul (cuvintele) de lungime maximă.

Exemplu: Dacă în fișier se găsește fraza "mie îmi plac caii, sunt fan după ei", atunci lista va fi: ("mie"->"îmi"->"plac"->"caii"->"sunt"->"fan"->"după"->"ei"), iar cuvintele de lungime maximă ce trebuie afișate sunt: "plac", "caii", "sunt" și "după".
14. Să se simuleze, folosind structuri dinamice de date, jocul "Gâsca roșie" jucat de doi jucători. Un pachet de 32 de cărți este împărțit în mod egal celor doi jucători. Cărțile se țin cu fața în jos, jucătorii pun alternativ pe masă cu fața în sus cartea care se află deasupra pachetului lor. Dacă s-a pus o carte roșie, celălalt jucător trebuie să ia întregul pachet de cărți de pe masă și-l pune sub pachetul său. Jocul continuă în acest fel până când unul din jucători rămâne fără cărți. Acesta câștigă jocul.